

CUSTOMER SUPPORT DATA ANALYSIS & TICKET RESOLUTION



Project Overview:

This project is about analyzing customer support data to understand and improve service quality.

We first separated all the tickets into three groups: resolved (closed), open, and tickets waiting for the customer's reply.

This helped us study each group better and understand different problems and delays.

We looked at what kind of issues were most common, how long they took to solve, and how happy the customers were.

We focused more on the resolved tickets to check what types took longer and which had low satisfaction ratings.

We used simple visualizations like bar charts and sunburst charts to make the patterns easy to understand.

This project helps support teams improve their speed and service to make customers happier.

Importing necessary libraries

```
In [1]: # For data manipulation and numerical operation
import pandas as pd
import numpy as np

# importing datetime for handling date and time conversions
```

```
from datetime import datetime

# For Data Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# importing warnings to ignore warning messages
import warnings
warnings.filterwarnings("ignore")

# For text processing
import re
import string
from collections import Counter
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.tokenize import TreebankWordTokenizer

#For Text Visualization
from wordcloud import WordCloud
```

In [2]: *# Downloading necessary resources for tokenizing and stopwords*

```
nltk.download('punkt')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package punkt to
[nltk_data]   C:\Users\dell\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]   C:\Users\dell\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

Out[2]: True

Data Loading

In [3]: `df = pd.read_csv("customer_support_tickets.csv")`

Initial Data Inspection

In [4]: `df.head()`

Out[4]:

	Ticket ID	Customer Name	Customer Email	Customer Age	Customer Gender	Product Purchased	Date
0	1	Marisa Obrien	carrollallison@example.com	32	Other	GoPro Hero	202
1	2	Jessica Rios	clarkeashley@example.com	42	Female	LG Smart TV	202
2	3	Christopher Robbins	gonzalestracy@example.com	48	Other	Dell XPS	202
3	4	Christina Dillon	bradleyolson@example.org	27	Female	Microsoft Office	202
4	5	Alexander Carroll	bradleymark@example.com	67	Female	Autodesk AutoCAD	202



Exploratory Data Analysis(EDA)

```
In [5]: print("Shape of the dataset =", df.shape)
```

Shape of the dataset = (8469, 17)

The dataset has a total of **8469 rows** and **17 columns**.

Each row represents one support ticket raised by a customer.

Each column contains useful information like customer details, ticket type, subject, response time, resolution time, and satisfaction rating.

This gives us a good amount of data to explore and understand how customer support is performing.

```
In [6]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8469 entries, 0 to 8468
Data columns (total 17 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   Ticket ID                            8469 non-null   int64
 1   Customer Name                        8469 non-null   object
 2   Customer Email                      8469 non-null   object
 3   Customer Age                        8469 non-null   int64
 4   Customer Gender                     8469 non-null   object
 5   Product Purchased                   8469 non-null   object
 6   Date of Purchase                    8469 non-null   object
 7   Ticket Type                         8469 non-null   object
 8   Ticket Subject                      8469 non-null   object
 9   Ticket Description                  8469 non-null   object
10   Ticket Status                      8469 non-null   object
11   Resolution                          2769 non-null   object
12   Ticket Priority                     8469 non-null   object
13   Ticket Channel                     8469 non-null   object
14   First Response Time                 5650 non-null   object
15   Time to Resolution                  2769 non-null   object
16   Customer Satisfaction Rating        2769 non-null   float64
dtypes: float64(1), int64(2), object(14)
memory usage: 1.1+ MB

```

The dataset contains a total of **8469 entries** and **17 columns**. It includes customer details, product information, ticket categories, and resolution-related data. Columns like **Customer Age** and **Ticket ID** are numerical, while others such as **Customer Name**, **Product Purchased**, **Ticket Type**, and **Ticket Channel** are categorical or textual. The **Ticket Description** column contains detailed text-based information explaining the customer's issue, which helps in understanding the nature of each support request. Some columns like **Resolution**, **First Response Time**, **Time to Resolution**, and **Customer Satisfaction Rating** have many missing values, indicating that not all tickets were resolved or rated. This overview helps us understand the structure, data types, and quality of the dataset before we begin further analysis.

```
In [7]: df.columns
```

```

Out[7]: Index(['Ticket ID', 'Customer Name', 'Customer Email', 'Customer Age',
              'Customer Gender', 'Product Purchased', 'Date of Purchase',
              'Ticket Type', 'Ticket Subject', 'Ticket Description', 'Ticket Status',
              'Resolution', 'Ticket Priority', 'Ticket Channel',
              'First Response Time', 'Time to Resolution',
              'Customer Satisfaction Rating'],
              dtype='object')

```

```
In [8]: df.loc[0]
```

```
Out[8]: Ticket ID
1
Customer Name Marisa Obri
en
Customer Email carrollallison@example.c
om
Customer Age
32
Customer Gender Oth
er
Product Purchased GoPro He
ro
Date of Purchase 2021-03-
22
Ticket Type Technical iss
ue
Ticket Subject Product set
up
Ticket Description I'm having an issue with the {product_purchas
e...
Ticket Status Pending Customer Respon
se
Resolution N
aN
Ticket Priority Critic
al
Ticket Channel Social med
ia
First Response Time 2023-06-01 12:15:
36
Time to Resolution N
aN
Customer Satisfaction Rating N
aN
Name: 0, dtype: object
```

```
In [9]: df.describe()
```

```
Out[9]:
```

	Ticket ID	Customer Age	Customer Satisfaction Rating
count	8469.000000	8469.000000	2769.000000
mean	4235.000000	44.026804	2.991333
std	2444.934048	15.296112	1.407016
min	1.000000	18.000000	1.000000
25%	2118.000000	31.000000	2.000000
50%	4235.000000	44.000000	3.000000
75%	6352.000000	57.000000	4.000000
max	8469.000000	70.000000	5.000000

This statistical summary helps us understand the distribution of numerical values in the dataset. The dataset includes **8469 customer support tickets**.

The **Customer Age** ranges from 18 to 70 years, with an average age of around 44. Most customers fall between the ages of 31 (25th percentile) and 57 (75th percentile), indicating that the majority of tickets are raised by adults across a wide age range.

The **Customer Satisfaction Rating** is available for 2769 resolved tickets only. The ratings range from 1 (lowest satisfaction) to 5 (highest satisfaction), with an average rating of approximately 3. This suggests a moderate level of satisfaction overall. Half of the ratings are 3 or below (median = 3), and 75% of them are 4 or below.

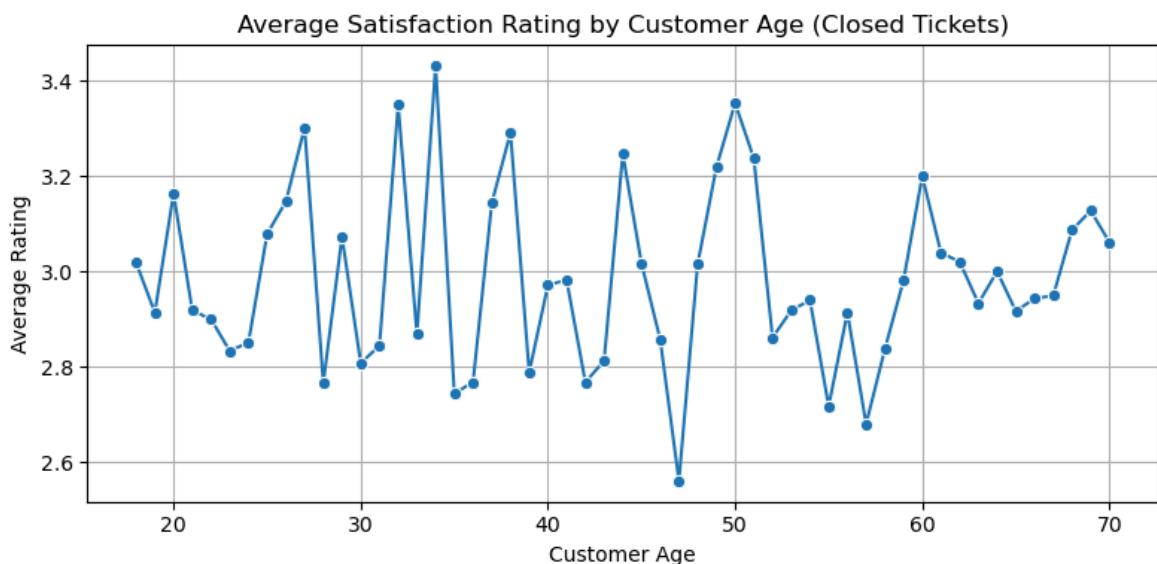
This summary provides a good foundation for understanding customer demographics and how satisfied they are with the support received.

Average Satisfaction Rating Across Age Groups for Resolved Tickets

```
In [10]: # Filtered closed tickets
closed_tickets = df[df['Ticket Status'] == 'Closed']

# Grouped by Customer Age and calculated average rating
avg_rating_by_age = closed_tickets.groupby('Customer Age')['Customer Satisfaction Rating'].mean()

# Plot
plt.figure(figsize=(8,4))
sns.lineplot(data=avg_rating_by_age, x='Customer Age', y='Customer Satisfaction Rating')
plt.title('Average Satisfaction Rating by Customer Age (Closed Tickets)')
plt.xlabel('Customer Age')
plt.ylabel('Average Rating')
plt.grid(True)
plt.tight_layout()
plt.show()
```



This line plot shows how the average **customer satisfaction rating** varies across different **age groups** for **closed (resolved) tickets**.

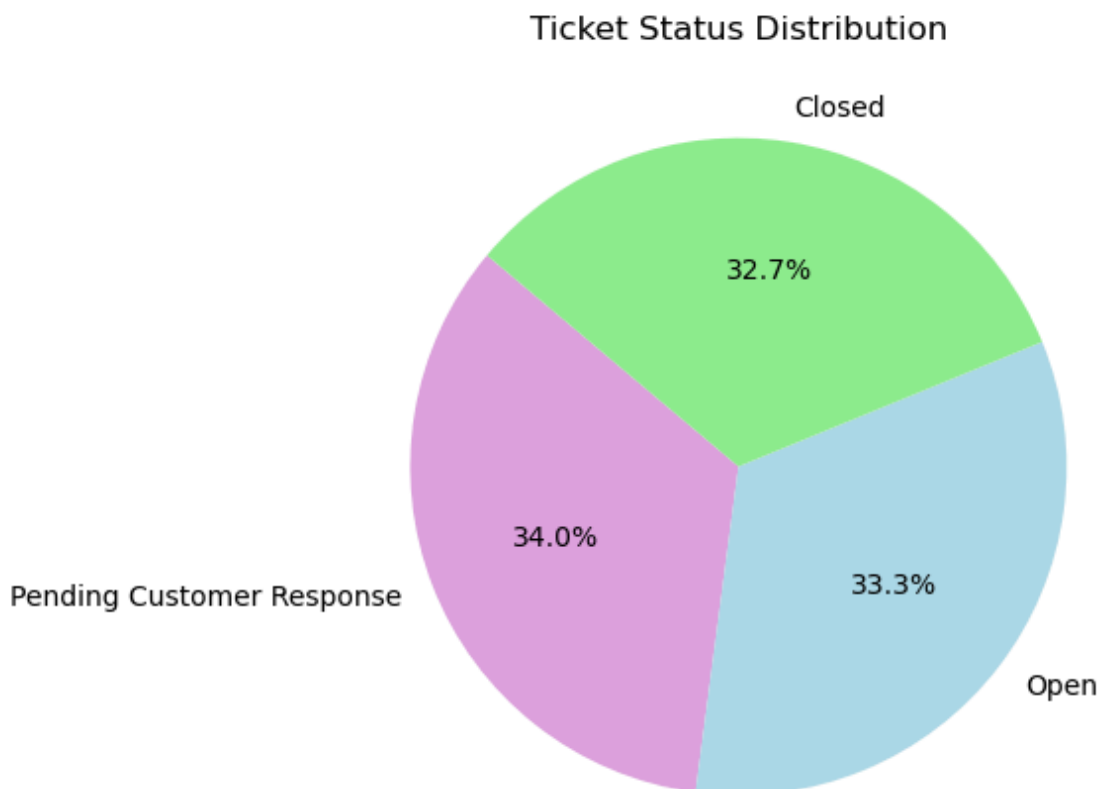
Ticket Status Distribution

```
In [11]: df['Ticket Status'].value_counts()
```

```
Out[11]: Ticket Status
Pending Customer Response    2881
Open                        2819
Closed                      2769
Name: count, dtype: int64
```

```
In [12]: statuses = ['Pending Customer Response', 'Open', 'Closed']
counts = [2881, 2819, 2769]

# Pie chart
plt.figure(figsize=(6, 6))
plt.pie(counts, labels=statuses, autopct='%1.1f%%', startangle=140, colors=['plu
plt.title('Ticket Status Distribution')
plt.tight_layout()
plt.show()
```



This pie chart shows the distribution of ticket statuses in the dataset. There are three types of statuses: **Pending Customer Response**, **Open**, and **Closed**. Each slice of the pie represents how many tickets are in each status, with percentages displayed on the chart. The colors used are **plum**, **light blue**, and

light green for better visualization. This helps us quickly understand how many tickets are still waiting for a customer reply, how many are currently open, and how many have been closed.

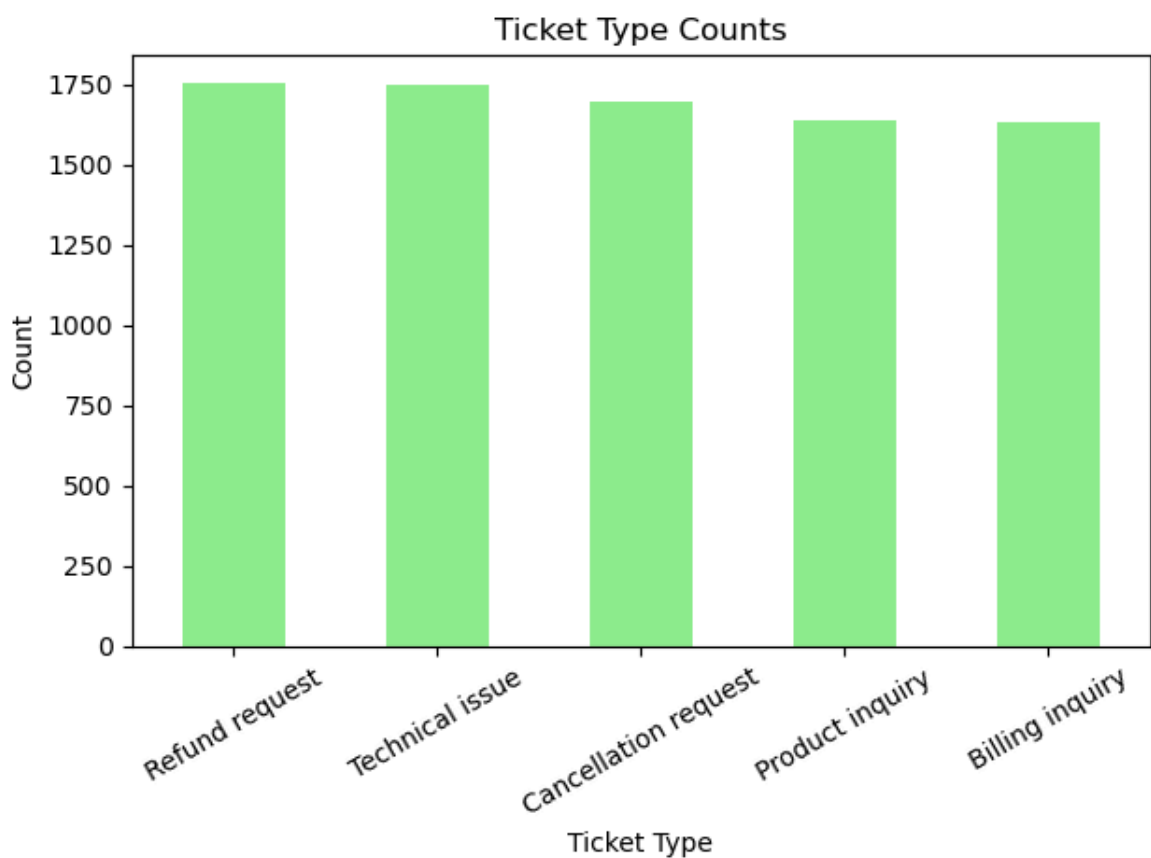
Ticket Type Distribution

```
In [13]: df['Ticket Type'].value_counts()
```

```
Out[13]: Ticket Type
Refund request      1752
Technical issue     1747
Cancellation request 1695
Product inquiry     1641
Billing inquiry     1634
Name: count, dtype: int64
```

```
In [14]: df['Ticket Type'].value_counts().plot(kind='bar', color='lightgreen')
```

```
plt.title('Ticket Type Counts')
plt.xlabel('Ticket Type')
plt.ylabel('Count')
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```



This bar chart shows how many tickets belong to each **Ticket Type**. The main types are **Refund request**, **Technical issue**, **Cancellation request**, **Product**

inquiry, and **Billing inquiry**. Each green bar's height tells us the number of tickets for that type. This helps us see which issues customers ask about the most.

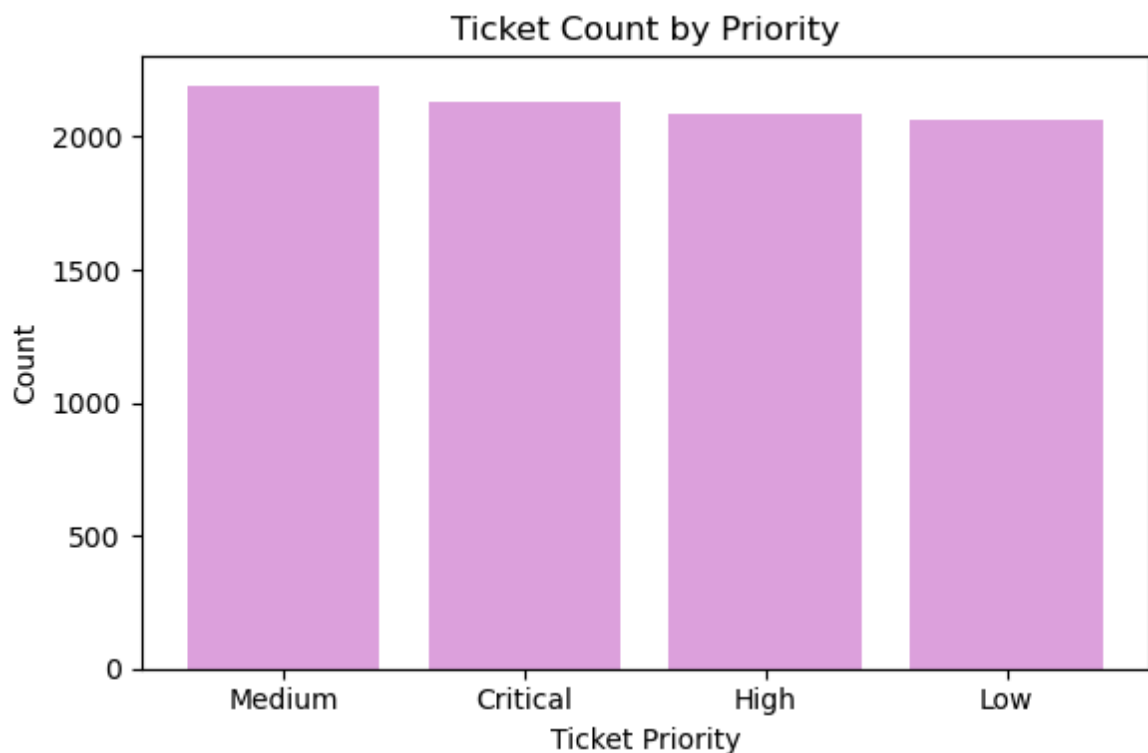
Ticket Count by Priority

```
In [15]: df['Ticket Priority'].value_counts()
```

```
Out[15]: Ticket Priority
Medium      2192
Critical     2129
High        2085
Low         2063
Name: count, dtype: int64
```

```
In [16]: # Data
priorities = ['Medium', 'Critical', 'High', 'Low']
counts = [2192, 2129, 2085, 2063]

# Plot
plt.figure(figsize=(6, 4))
plt.bar(priorities, counts, color='plum')
plt.xlabel('Ticket Priority')
plt.ylabel('Count')
plt.title('Ticket Count by Priority')
plt.tight_layout()
plt.show()
```



This bar chart shows the number of tickets for each **Ticket Priority** level. The priorities are **Medium**, **Critical**, **High**, and **Low**. Each purple bar represents

how many tickets fall into each priority category. This helps us understand which priority levels have more tickets that need attention.

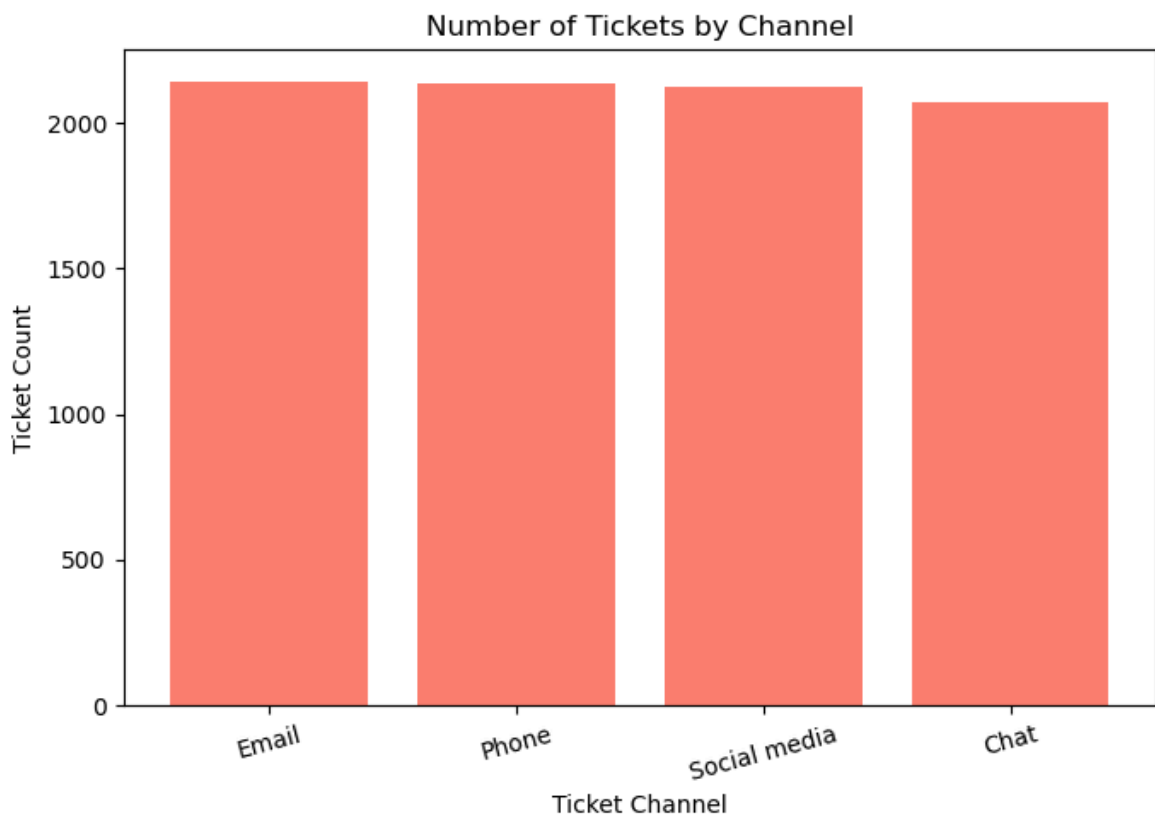
Ticket Distribution by Communication Channel

```
In [17]: df['Ticket Channel'].value_counts()
```

```
Out[17]: Ticket Channel
Email          2143
Phone          2132
Social media   2121
Chat           2073
Name: count, dtype: int64
```

```
In [18]: # Data
channels = ['Email', 'Phone', 'Social media', 'Chat']
counts = [2143, 2132, 2121, 2073]

# Plot
plt.figure(figsize=(7, 5))
plt.bar(channels, counts, color='salmon')
plt.xlabel('Ticket Channel')
plt.ylabel('Ticket Count')
plt.title('Number of Tickets by Channel')
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
```



This bar chart shows how many tickets were received through different **channels** such as **Email**, **Phone**, **Social media**, and **Chat**. Each **salmon-colored** bar represents the number of tickets from each channel. This helps us see which communication method customers used the most.

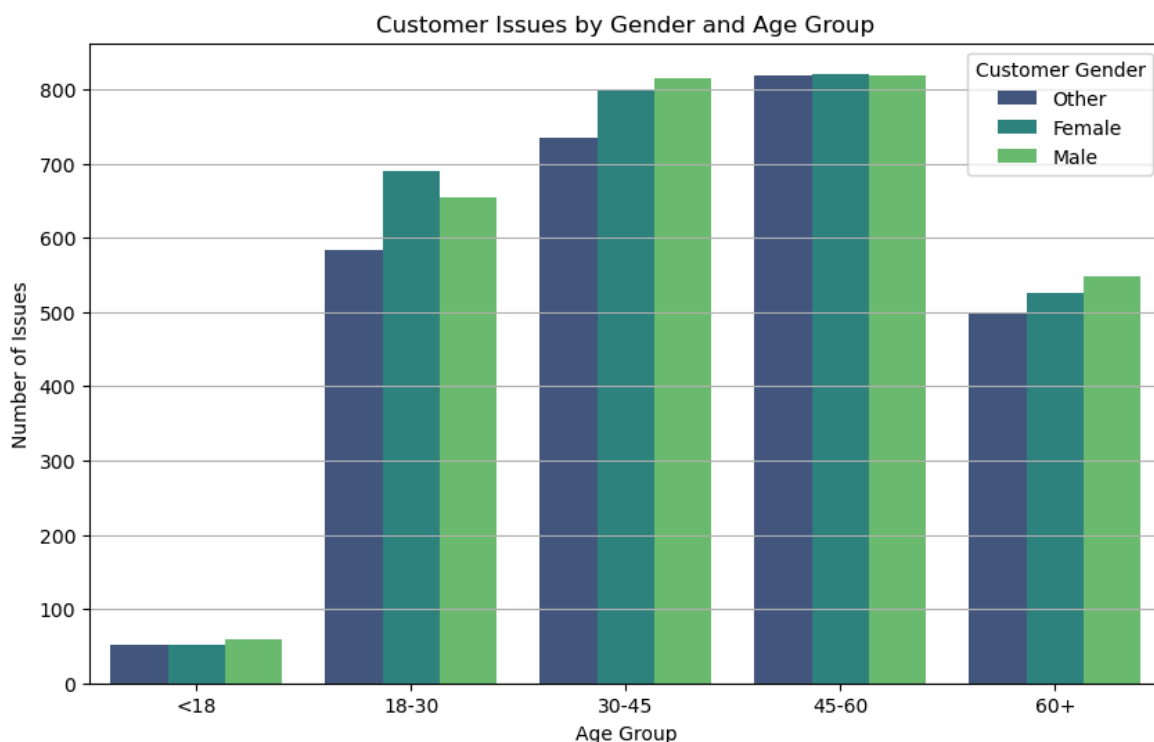
Customer Issues by Gender and Age Group

```
In [19]: df['Customer Gender'].value_counts()
```

```
Out[19]: Customer Gender
Male      2896
Female    2887
Other     2686
Name: count, dtype: int64
```

```
In [20]: # Creating age bins
df['Age Group'] = pd.cut(df['Customer Age'], bins=[0, 18, 30, 45, 60, 100],
                        labels=['<18', '18-30', '30-45', '45-60', '60+'])

# Plotting count of problems by Gender and Age Group
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x='Age Group', hue='Customer Gender', palette='viridis')
plt.title('Customer Issues by Gender and Age Group')
plt.xlabel('Age Group')
plt.ylabel('Number of Issues')
plt.grid(True, axis='y')
plt.show()
```



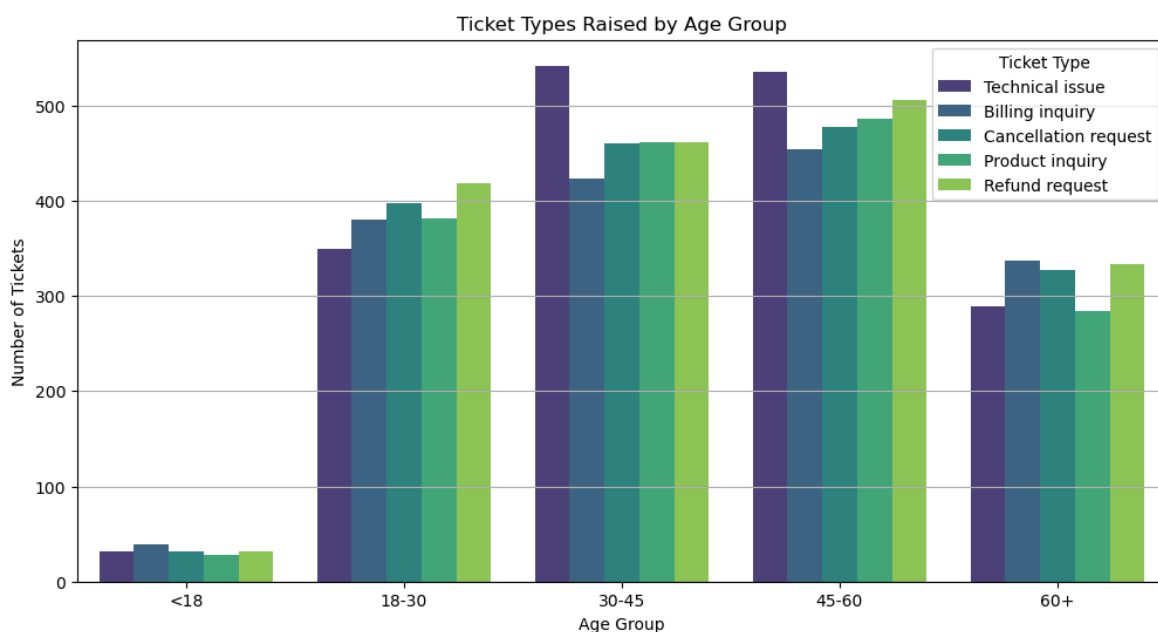
This bar chart shows how many customer issues came from different **age groups** and how these are split by **gender**. Age groups include **under 18**, **18–**

30, 30–45, 45–60, and 60+. Each bar is colored by **Customer Gender**, so we can compare the number of issues reported by males and females in each age group. This helps us understand which age and gender combinations report more issues.

Types of Issues Reported by Different Age Groups

```
In [21]: # Creating age bins
df['Age Group'] = pd.cut(df['Customer Age'], bins=[0, 18, 30, 45, 60, 100],
                        labels=['<18', '18-30', '30-45', '45-60', '60+'])

# Plotting Ticket Type vs Age Group
plt.figure(figsize=(12, 6))
sns.countplot(data=df, x='Age Group', hue='Ticket Type', palette='viridis')
plt.title('Ticket Types Raised by Age Group')
plt.xlabel('Age Group')
plt.ylabel('Number of Tickets')
plt.legend(title='Ticket Type')
plt.grid(True, axis='y')
plt.show()
```



This bar chart shows which **age groups** raise different types of **tickets** or problems. The age groups include **under 18, 18–30, 30–45, 45–60, and 60+.** Each colored bar represents a different **Ticket Type**, such as technical issues, refund requests, or cancellations. From the chart, we can see that most of the problems are reported by people aged **30–45** and **45–60**. Among these, **Technical issues** are the most common. This helps us understand which age groups face more technical problems.

Status Counts for Each Kind of Support Ticket

```
In [22]: ticket_types = df['Ticket Type'].unique()
          statuses = ['Closed', 'Open', 'Pending Customer Response']

          for ticket_type in ticket_types:
              print(f"\nTicket Type: {ticket_type}")
              for status in statuses:
                  count = df[(df['Ticket Type'] == ticket_type) & (df['Ticket Status'] ==
                  print(f"    {status}: {count}")
```

Ticket Type: Technical issue
Closed: 580
Open: 602
Pending Customer Response: 565

Ticket Type: Billing inquiry
Closed: 544
Open: 539
Pending Customer Response: 551

Ticket Type: Cancellation request
Closed: 516
Open: 582
Pending Customer Response: 597

Ticket Type: Product inquiry
Closed: 533
Open: 532
Pending Customer Response: 576

Ticket Type: Refund request
Closed: 596
Open: 564
Pending Customer Response: 592

This summary shows how each **Ticket Type** is spread across different **statuses** like **Closed**, **Open**, and **Pending Customer Response**. For example, **Technical issues** have almost equal counts across all three statuses, while **Cancellation requests** and **Refund requests** also show high numbers in the **Pending** category. This helps us understand the progress and resolution stage of each type of customer problem.

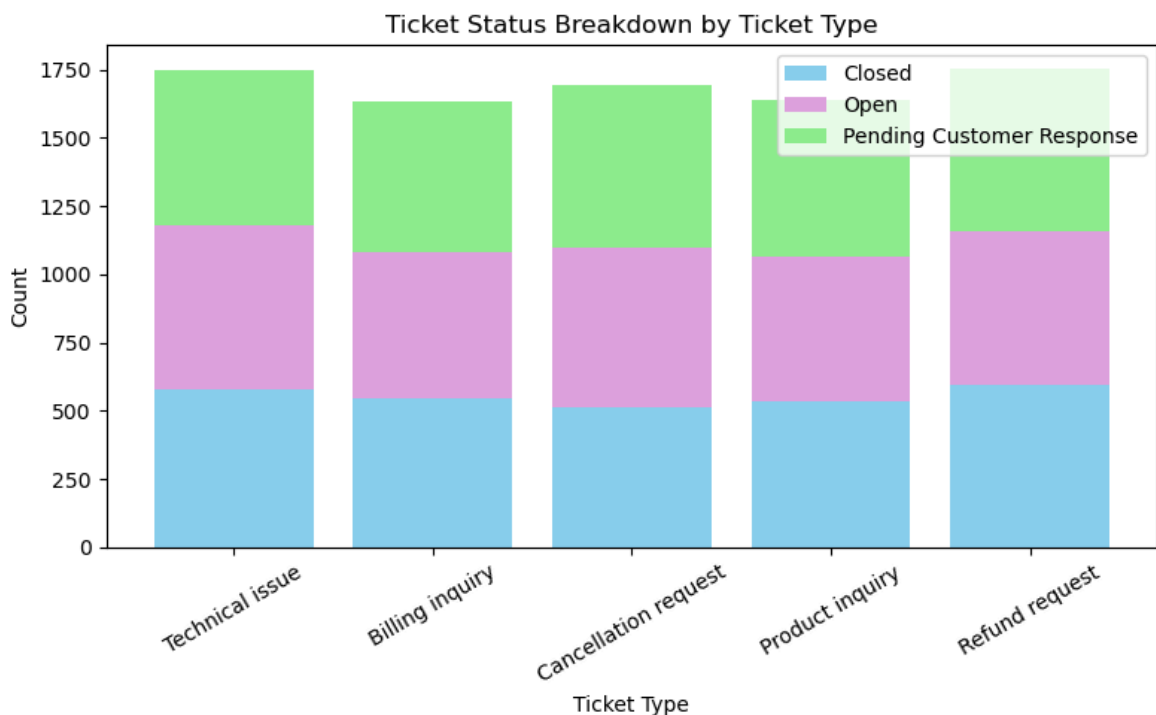
Status Distribution for Ticket Type Issue

```
In [23]: # Data
          ticket_types = ['Technical issue', 'Billing inquiry', 'Cancellation request', 'P
          closed = [580, 544, 516, 533, 596]
          open_status = [602, 539, 582, 532, 564]
          pending = [565, 551, 597, 576, 592]
```

```
# Creating a stacked bar plot
plt.figure(figsize=(8, 5))

plt.bar(ticket_types, closed, label='Closed', color='skyblue')
plt.bar(ticket_types, open_status, bottom=closed, label='Open', color='plum')
plt.bar(ticket_types, pending, bottom=[i+j for i,j in zip(closed, open_status)],

plt.title('Ticket Status Breakdown by Ticket Type')
plt.xlabel('Ticket Type')
plt.ylabel('Count')
plt.xticks(rotation=30)
plt.legend()
plt.tight_layout()
plt.show()
```



This stacked bar chart shows how different **ticket types** are divided into **Closed**, **Open**, and **Pending Customer Response** statuses. Each bar is split into three colored sections, helping us easily compare the progress of each type. For example, **Technical issues** Refund requests have many tickets still **Open** or **Pending**. This chart gives a clear view of how well different issues are being resolved.

Ticket Priority Breakdown by Ticket Type

```
In [24]: ticket_types = df['Ticket Type'].unique()
priorities = ['Low', 'Medium', 'High', 'Critical']

for ticket_type in ticket_types:
    print(f"\nTicket Type: {ticket_type}")
    for priority in priorities:
```

```
count = df[(df['Ticket Type'] == ticket_type) & (df['Ticket Priority'] =
print(f" Priority {priority}: {count}")
```

Ticket Type: Technical issue

Priority Low: 413
Priority Medium: 437
Priority High: 458
Priority Critical: 439

Ticket Type: Billing inquiry

Priority Low: 398
Priority Medium: 434
Priority High: 382
Priority Critical: 420

Ticket Type: Cancellation request

Priority Low: 414
Priority Medium: 460
Priority High: 398
Priority Critical: 423

Ticket Type: Product inquiry

Priority Low: 398
Priority Medium: 441
Priority High: 399
Priority Critical: 403

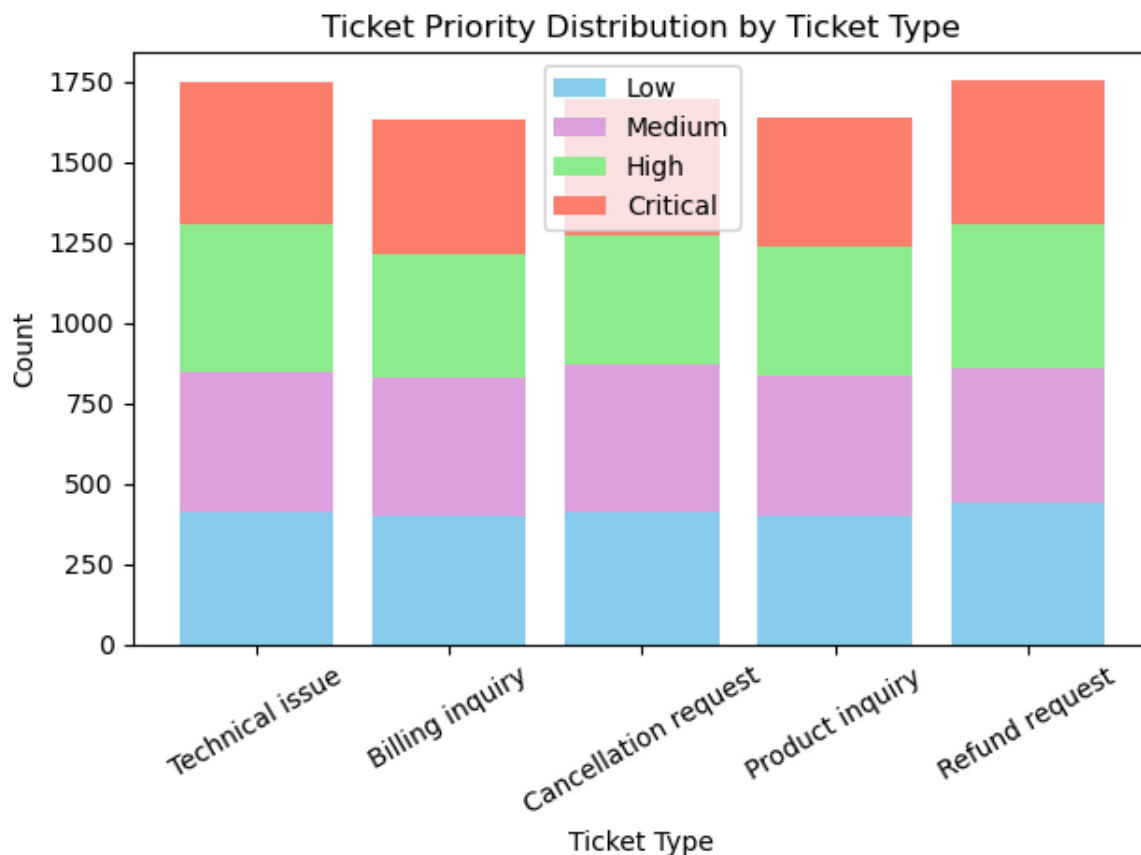
Ticket Type: Refund request

Priority Low: 440
Priority Medium: 420
Priority High: 448
Priority Critical: 444

```
In [25]: # Data
ticket_types = ['Technical issue', 'Billing inquiry', 'Cancellation request', 'P
low = [413, 398, 414, 398, 440]
medium = [437, 434, 460, 441, 420]
high = [458, 382, 398, 399, 448]
critical = [439, 420, 423, 403, 444]

# Plotting
plt.bar(ticket_types, low, label='Low', color='skyblue')
plt.bar(ticket_types, medium, bottom=low, label='Medium', color='plum')
plt.bar(ticket_types, high, bottom=[i+j for i,j in zip(low, medium)], label='Hig
plt.bar(ticket_types, critical, bottom=[i+j+k for i,j,k in zip(low, medium, high

plt.xlabel('Ticket Type')
plt.ylabel('Count')
plt.title('Ticket Priority Distribution by Ticket Type')
plt.xticks(rotation=30)
plt.legend()
plt.tight_layout()
plt.show()
```



This summary shows how different **ticket types** are spread across **priority levels** — **Low**, **Medium**, **High**, and **Critical**. For example, **Technical issues** and **Refund requests** have a high number of **High** and **Critical** priority tickets, meaning they are considered more urgent. On the other hand, tickets like **Billing inquiry** and **Product inquiry** also have a balanced spread across all priorities. This helps us understand how serious or urgent different types of problems are.

Ticket Resolution and Urgency Overview by Category

```
In [26]: ticket_types = df['Ticket Type'].unique()
priorities = ['Low', 'Medium', 'High', 'Critical']
statuses = ['Closed', 'Open', 'Pending Customer Response']

for ticket_type in ticket_types:
    print(f"\nTicket Type: {ticket_type}")
    for priority in priorities:
        print(f"  Priority: {priority}")
        for status in statuses:
            count = df[
                (df['Ticket Type'] == ticket_type) &
                (df['Ticket Priority'] == priority) &
                (df['Ticket Status'] == status)
            ].shape[0]
            print(f"    {status}: {count}")
```

Ticket Type: Technical issue

Priority: Low
Closed: 138
Open: 151
Pending Customer Response: 124
Priority: Medium
Closed: 138
Open: 153
Pending Customer Response: 146
Priority: High
Closed: 157
Open: 152
Pending Customer Response: 149
Priority: Critical
Closed: 147
Open: 146
Pending Customer Response: 146

Ticket Type: Billing inquiry

Priority: Low
Closed: 133
Open: 137
Pending Customer Response: 128
Priority: Medium
Closed: 133
Open: 131
Pending Customer Response: 170
Priority: High
Closed: 135
Open: 134
Pending Customer Response: 113
Priority: Critical
Closed: 143
Open: 137
Pending Customer Response: 140

Ticket Type: Cancellation request

Priority: Low
Closed: 105
Open: 143
Pending Customer Response: 166
Priority: Medium
Closed: 142
Open: 154
Pending Customer Response: 164
Priority: High
Closed: 124
Open: 151
Pending Customer Response: 123
Priority: Critical
Closed: 145
Open: 134
Pending Customer Response: 144

Ticket Type: Product inquiry

Priority: Low
Closed: 118
Open: 134
Pending Customer Response: 146
Priority: Medium

Closed: 140
Open: 145
Pending Customer Response: 156
Priority: High
Closed: 131
Open: 131
Pending Customer Response: 137
Priority: Critical
Closed: 144
Open: 122
Pending Customer Response: 137

Ticket Type: Refund request

Priority: Low
Closed: 150
Open: 131
Pending Customer Response: 159
Priority: Medium
Closed: 141
Open: 144
Pending Customer Response: 135
Priority: High
Closed: 158
Open: 136
Pending Customer Response: 154
Priority: Critical
Closed: 147
Open: 153
Pending Customer Response: 144

This detailed breakdown presents how each **ticket type** is distributed across various **priority levels (Low, Medium, High, Critical)** and **ticket statuses (Closed, Open, Pending Customer Response)**. It shows the resolution trends and urgency patterns of different customer issues.

For example, **Technical issues** and **Refund requests** show higher numbers in the **High** and **Critical** categories, indicating urgency and critical attention needed. **Cancellation requests** have more **Pending Customer Response** statuses, especially in **Low** and **Medium** priorities, which could signal a need for better follow-up processes. **Billing inquiry** where priority is **Medium** also shows a high number of **Pending Customer Response**.

This summary helps in identifying which **issue types** often remain **unresolved**, their **urgency level**, and where **resource allocation** or **customer engagement** can be improved.

Ticket Status Distribution by Ticket Priority

```
In [27]: priorities = df['Ticket Priority'].unique()
        statuses = ['Closed', 'Open', 'Pending Customer Response']

        for priority in priorities:
            print(f"\nTicket Priority: {priority}")
            for status in statuses:
                count = df[(df['Ticket Priority'] == priority) & (df['Ticket Status'] == status)].count()
                print(f"    {status}: {count}")
```

Ticket Priority: Critical
 Closed: 726
 Open: 692
 Pending Customer Response: 711

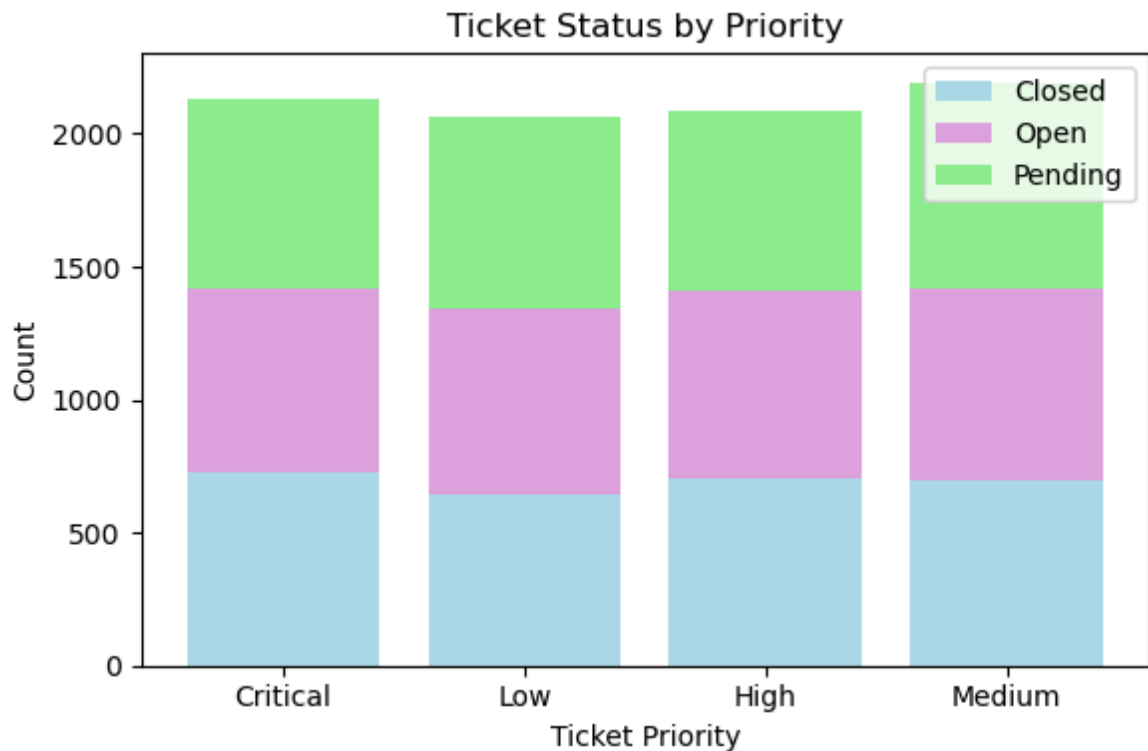
Ticket Priority: Low
 Closed: 644
 Open: 696
 Pending Customer Response: 723

Ticket Priority: High
 Closed: 705
 Open: 704
 Pending Customer Response: 676

Ticket Priority: Medium
 Closed: 694
 Open: 727
 Pending Customer Response: 771

```
In [28]: # Data
        priorities = ['Critical', 'Low', 'High', 'Medium']
        closed = [726, 644, 705, 694]
        open_ = [692, 696, 704, 727]
        pending = [711, 723, 676, 771]

        # Plot
        plt.figure(figsize=(6, 4))
        plt.bar(priorities, closed, label='Closed', color='lightblue')
        plt.bar(priorities, open_, bottom=closed, label='Open', color='plum')
        plt.bar(priorities, pending, bottom=[closed[i] + open_[i] for i in range(len(priorities))], label='Pending', color='lightcoral')
        plt.xlabel('Ticket Priority')
        plt.ylabel('Count')
        plt.title('Ticket Status by Priority')
        plt.legend()
        plt.tight_layout()
        plt.show()
```



Missing Data Summary in the Dataset

```
In [29]: df.isnull().sum()
```

```
Out[29]: Ticket ID          0
Customer Name         0
Customer Email        0
Customer Age          0
Customer Gender       0
Product Purchased     0
Date of Purchase      0
Ticket Type           0
Ticket Subject        0
Ticket Description    0
Ticket Status         0
Resolution            5700
Ticket Priority        0
Ticket Channel        0
First Response Time   2819
Time to Resolution    5700
Customer Satisfaction Rating 5700
Age Group             0
dtype: int64
```

The dataset has no missing values in columns like **Ticket ID**, **Customer Name**, **Customer Age**, and **Ticket Status**. However, several important columns have many missing values: **Resolution**, **Time to Resolution**, and **Customer Satisfaction Rating** each have **5700 missing entries**. Also, **First Response Time** has **2819 missing entries**. This indicates that many tickets may not have been resolved or rated yet, affecting the completeness of some analyses.

```
In [30]: # Checking missing values count and percentage
missing_count = df.isnull().sum()
missing_percent = (missing_count / len(df)) * 100

# Combining into one DataFrame for clear view
missing_df = pd.DataFrame({
    'Missing Count': missing_count,
    'Missing %': missing_percent
})

# Filtering only columns with missing values
missing_df = missing_df[missing_df['Missing Count'] > 0].sort_values(by='Missing

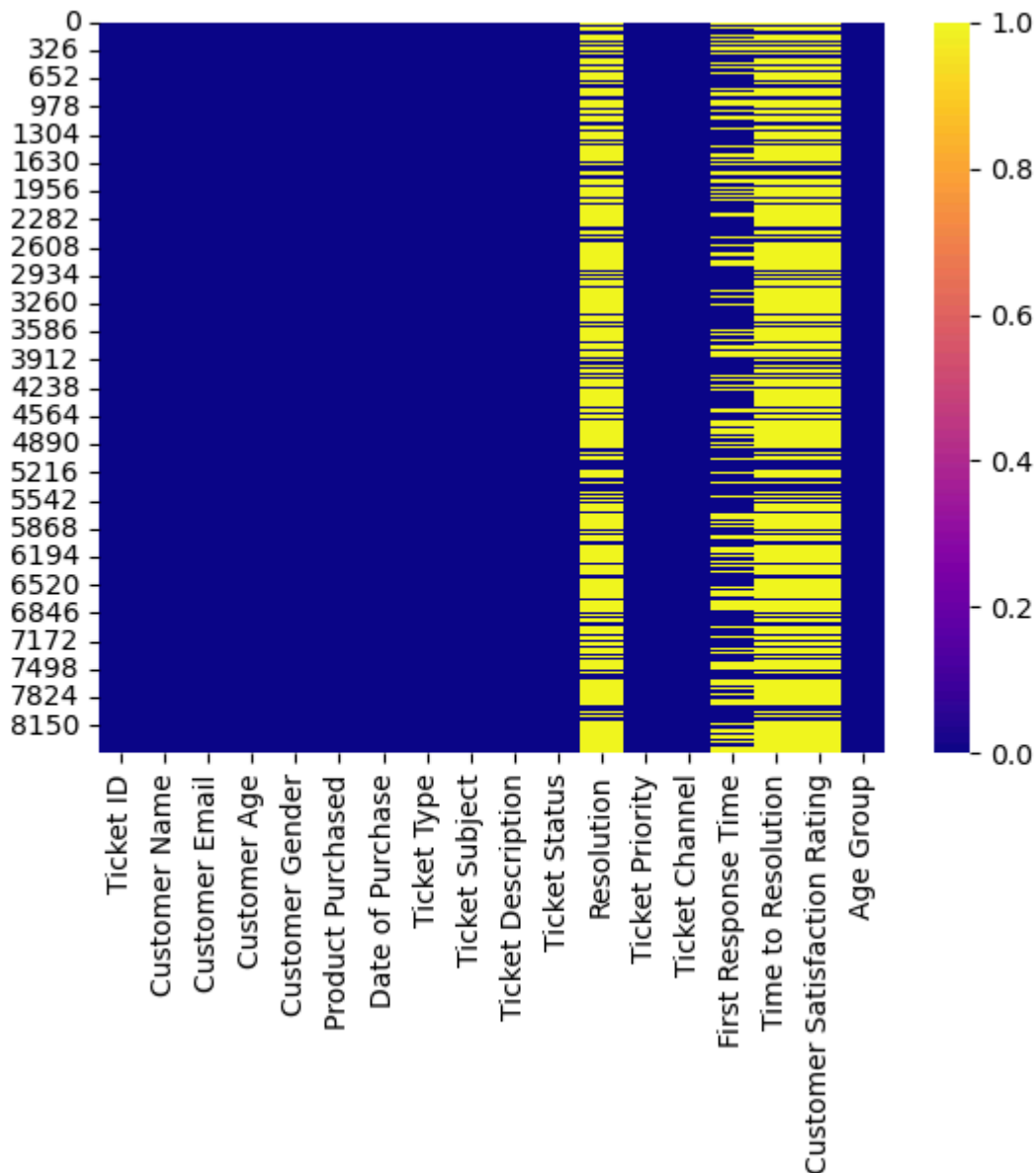
# Showing the result
print(missing_df)
```

	Missing Count	Missing %
Resolution	5700	67.304286
Time to Resolution	5700	67.304286
Customer Satisfaction Rating	5700	67.304286
First Response Time	2819	33.286102

Visual Representation Of Missing Values

```
In [31]: sns.heatmap(df.isnull(), cmap='plasma')
```

```
Out[31]: <Axes: >
```



DateTime Conversion

```
In [32]: # converting date columns to datetime format
df['Date of Purchase'] = pd.to_datetime(df['Date of Purchase'])
df['First Response Time'] = pd.to_datetime(df['First Response Time'], errors='coerce')
df['Time to Resolution'] = pd.to_datetime(df['Time to Resolution'], errors='coerce')
```

In this step, I converted the 'Date of Purchase', 'First Response Time', and 'Time to Resolution' columns into datetime format. This ensures that I can perform time-based operations like calculating delays and resolution times.

Seperating Data into Resolved and Unresolved

```
In [33]: df['Is Resolved'] = df['Time to Resolution'].notnull()
```

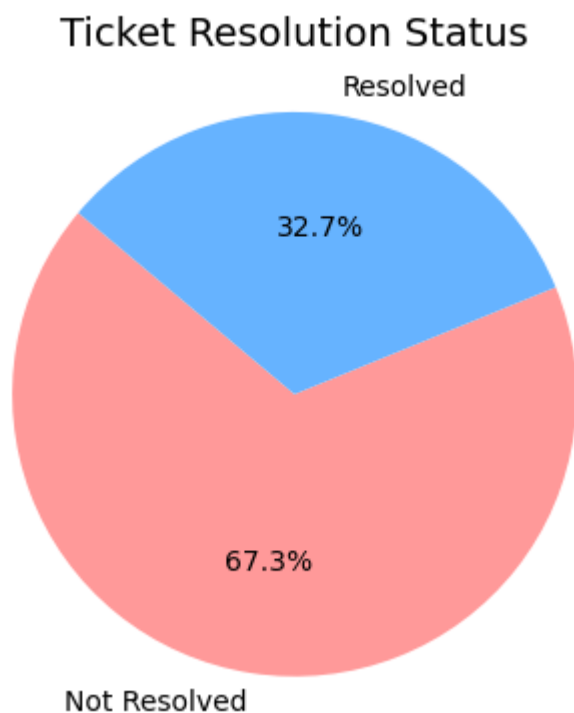
```
In [34]: df['Is Resolved'].value_counts()
```

```
Out[34]: Is Resolved
False    5700
True     2769
Name: count, dtype: int64
```

```
In [35]: #plotting the resolved and unresolved data
resolved_counts = df['Is Resolved'].value_counts()

labels = ['Not Resolved', 'Resolved']

# Pie chart
plt.figure(figsize=(4, 4))
plt.pie(resolved_counts,
        labels=labels,
        autopct='%1.1f%%',
        colors=['#ff9999', '#66b3ff'],
        startangle=140)
plt.title('Ticket Resolution Status', y=1.04, fontsize=14)
plt.axis('equal')
plt.show()
```



```
In [36]: df[df['Is Resolved'] == False]['Ticket Subject'].value_counts().head(20)
```

```
Out[36]: Ticket Subject
Refund request      398
Delivery problem    383
Software bug        375
Installation support 372
Product compatibility 372
Payment issue       370
Battery life        369
Hardware issue      364
Product setup       346
Data loss           344
Cancellation request 339
Peripheral compatibility 338
Account access      338
Network problem     338
Product recommendation 331
Display issue       323
Name: count, dtype: int64
```

Text Cleaning and Preprocessing

```
In [37]: tokenizer = TreebankWordTokenizer()
stop_words = set(stopwords.words('english'))

def clean_text(text):
    if pd.isnull(text):
        return ""
    text = text.lower()
    text = re.sub(r'^a-zA-Z\s', '', text)
    words = tokenizer.tokenize(text)
    words = [word for word in words if word not in stop_words]
    return ' '.join(words)

df['Cleaned_Description'] = df['Ticket Description'].apply(clean_text)
```

```
In [38]: # Combined all cleaned text
all_words = ' '.join(df['Cleaned_Description']).split()

# Getting most common words
word_freq = Counter(all_words)
print(word_freq.most_common(20))
```

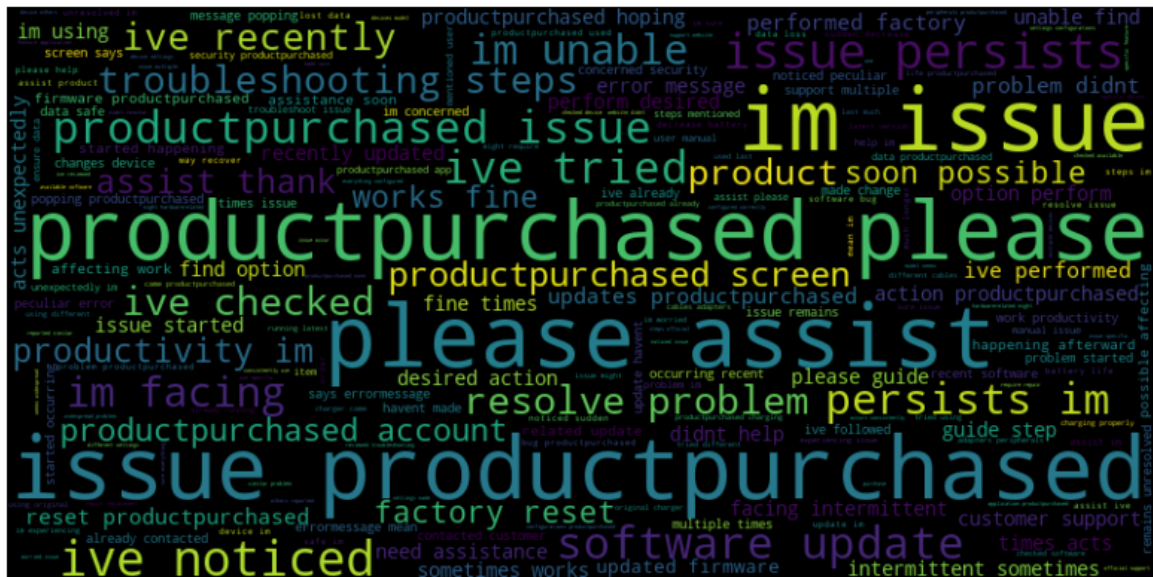
```
[('productpurchased', 14048), ('issue', 11530), ('im', 10287), ('please', 8808),
('assist', 6250), ('ive', 6001), ('problem', 2285), ('product', 2270), ('data', 1
654), ('software', 1552), ('account', 1461), ('steps', 1390), ('noticed', 1208),
('persists', 1178), ('help', 1176), ('resolve', 1165), ('update', 1155), ('woul
d', 1122), ('device', 1106), ('unable', 1088)]
```

Word Cloud of Cleaned Ticket Descriptions

```
In [39]: wordcloud = WordCloud(width=800, height=400, background_color='black').generate(

plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
```

```
plt.axis('off')
plt.show()
```



Calculating Response Delay and Resolution Time in Hours

```
In [40]: df['Response Delay (in hours)'] = (df['First Response Time'] - df['Date of Purchase'])
df['Resolution Time (in hours)'] = (df['Time to Resolution'] - df['First Response Time'])
```

I created two new columns ... 'Response Delay' to track the time between purchase and first response, and 'Resolution Time' to measure the time between first response and issue resolution. Both help in analyzing the support team's response and resolution speed.

Frequency of Resolved Tickets by Ticket Type

```
In [41]: # Filter resolved tickets with valid (positive) resolution time
resolved_tickets = df[(df['Is Resolved'] == True) & (df['Resolution Time (in hou

# Count the frequency of each Ticket Type
resolved_type_counts = resolved_tickets['Ticket Type'].value_counts()

# Display
print("Resolved Ticket Type Frequency:")
print(resolved_type_counts)
```

Resolved Ticket Type Frequency:

Ticket Type	
Technical issue	305
Refund request	303
Billing inquiry	273
Cancellation request	265
Product inquiry	256

Name: count, dtype: int64

Frequency of Active Unresolved Tickets by Ticket Type (Excluding Pending Customer Response) / for Open Ticket

```
In [42]: # Filtering unresolved tickets where Time to Resolution is NaT and First Response is not NaT
unresolved_pending_customer = df[
    (df['Is Resolved'] == False) &
    (df['Time to Resolution'].isna()) &
    (df['First Response Time'].notna())
]

# Now excluding them to focus on active unresolved tickets
active_unresolved = df[
    (df['Is Resolved'] == False) &
    (~df.index.isin(unresolved_pending_customer.index))
]

# Counting Ticket Types
unresolved_type_counts = active_unresolved['Ticket Type'].value_counts()

# Displaying
print("\nUnresolved Ticket Type Frequency (excluding pending customer reply):")
print(unresolved_type_counts)
```

Unresolved Ticket Type Frequency (excluding pending customer reply):

Ticket Type	
Technical issue	602
Cancellation request	582
Refund request	564
Billing inquiry	539
Product inquiry	532

Name: count, dtype: int64

Proceeding To Handle Data Where Customer Response is Pending

```
In [43]: # Checking how many days customer has not responded after support replied, to au

# Assuming today's date for reference
today = pd.to_datetime("today")

# Calculating how long customer hasn't responded (only if resolution is NaT)
df['Customer_Response_Pending_Days'] = df.apply(
    lambda row: (today - row['First Response Time']).days if pd.isna(row['Time to Resolution']) else 0,
    axis=1
)
```

```
)  
  
# Marking tickets for auto-closing if no customer response in 90+ days  
df['Auto_Close'] = df['Customer_Response_Pending_Days'].apply(lambda x: True if
```

In [44]: *# Getting all tickets where customer didn't respond for over 90 days*

```
auto_close_tickets = df[df['Auto_Close'] == True]
```

In [45]: *# Removing tickets with negative resolution time*

```
df_filtered = df[df['Resolution Time (in hours)'] >= 0]
```

Data Saved into a New CSV file

In [46]: *# Counting tickets to be auto-closed*

```
print("Tickets to auto-close:", auto_close_tickets.shape[0])
```

Tickets to auto-close: 2881

In [47]: *# Exporting auto-closable tickets to CSV*

```
auto_close_tickets.to_csv("auto_close_tickets.csv", index=False)
```

In [48]: `df.shape`

Out[48]: (8469, 24)

Resolved data with Positive Resolution Time (in hours)

In [49]:

```
closed_positive_res_time = df[(df['Time to Resolution'].notna()) & (df['Resoluti  
print("Number of closed tickets with positive resolution time:", closed_positive
```

Number of closed tickets with positive resolution time: 1402

We found that after calculating the resolution time in hours, some data have negative resolution times. This might be due to incorrect data entry or differences in time zones.

In [50]: *# Filtering the dataframe where Ticket Status is 'Open'*

```
open_tickets_df = df[df['Ticket Status'] == 'Open']
```

Saved this filtered data to CSV

```
open_tickets_df.to_csv('open_tickets.csv', index=False)
```

```
print(f"Saved {len(open_tickets_df)} rows where Ticket Status is 'Open' to open_
```

Saved 2819 rows where Ticket Status is 'Open' to open_tickets.csv

In [51]: *#Saving the data of resolved ticket to a new CSV*

```
resolved_tickets = df[df['Is Resolved'] == True]
```

```
resolved_tickets.to_csv('resolved_tickets.csv', index=False)  
print(f"Saved {resolved_tickets.shape[0]} resolved tickets to 'resolved_tickets.'")
```

Saved 2769 resolved tickets to 'resolved_tickets.csv'

How Can We Improve Response Time for Customer Ticket Data Analysis?

Improving response time helps build trust and increases customer satisfaction. Below are several suggestions with their explanations:

1. Analyze Ticket Priority and Status Carefully

We noticed that ticket types like **Technical Issue** and **Refund Request** mostly appear in *High* and *Critical* priority. So, we must assign more skilled agents and faster channels (like phone or chatbot) to such tickets. This reduces waiting time and improves customer trust.

2. Build a Smart Chatbot for Common Queries

Many tickets are simple and repetitive. For example, "How do I reset my password?" or "Where can I track my order?" A chatbot can instantly answer these without human help. If the customer is not satisfied, the chatbot can connect them to live agents.

3. Improve Help Section with Pre-Answered FAQs

Add a clearly visible help section with common questions and answers. If users can find answers themselves, they won't create new tickets. This reduces overall ticket volume and allows support teams to focus on urgent problems.

4. Handle Elderly Customers with Care

From our data, we noticed many tickets were created by elderly users. They may struggle with technology. So, we should offer them priority service, a dedicated helpline, and simplified instructions. Also, friendly tone and patient agents make a big difference.

5. Monitor Resolution and Response Delays

We created two columns: **Response Delay (in hours)** and **Resolution Time (in hours)**. This helped us find tickets that were answered too late. We should focus on those agents or ticket types that usually take longer and improve the process or training.

6. Use Automation to Close Unresponsive Tickets

In the **auto_close_tickets.csv** file, we found tickets where the **customer support team has already replied** to the customer's issue, but the **customer did not reply back** after that.

To handle such cases, we decided to set a **response deadline of 90 days**. If the customer does not respond within these 90 days, we **automatically mark the ticket as resolved** and close it.

This helps us keep the system clean and avoids waiting on tickets that are likely no longer active. If the customer still needs help, they can always **raise a new ticket later** and get support again.

7. Check Data Quality – Watch for Errors

In **resolved_tickets.csv**, some tickets showed negative resolution times. This might be due to wrong data entry or different time zones. We should set up checks to flag and fix such errors in real time to ensure accurate reporting.

8. Focus on Unresolved Tickets Without Response

In **open_tickets.csv**, tickets were left without any reply from support. These need immediate attention. Managers should review them regularly and assign them higher priority to avoid customer dissatisfaction.

Conclusion

After analyzing the entire dataset, we categorized tickets into three groups:

- ✓ **Resolved Tickets:** Closed successfully, though some had issues like wrong timestamps.
- ✓ **Unresolved Tickets:** Still open without a solution — need urgent action.
- ✓ **Pending Customer Response:** Support team has replied, but waiting for customer's reply. These are set to auto-close after 90 days.

Our main focus should be on:

- Responding quicker to **High/Critical** priority issues
- Assisting **elderly customers** better
- Using **chatbots** and **FAQs** to reduce repetitive tickets
- Regularly checking for **data errors** and cleaning up inactive tickets
- Ensuring no ticket goes **unattended** for long

In []: